

Rheinische
Hochschule
Köln

Fachbereich Ingenieurwesen

Studiengang Informatik (B.Sc.)

Praxistransferprojekt

Aufbau und Evaluation einer GitOps-basierten Kubernetes-Umgebung auf AWS mit k3s durch kontrollierte Fehlerexperimente

vorgelegt von
Konstantin Muschallik
Matrikelnr. 653242001

Wintersemester 2026/2027



Rheinische
Hochschule
Köln

Fachbereich Ingenieurwesen

Studiengang Informatik (B.Sc.)

Praxistransferprojekt

Aufbau und Evaluation einer GitOps-basierten Kubernetes-Umgebung auf AWS mit k3s durch kontrollierte Fehlerexperimente

vorgelegt von
Konstantin Muschallik
Matrikelnr. 653242001

Erstprüfer: Prof. Dr. Johannes Mauer

Wintersemester 2026/2027

Inhaltsverzeichnis

1. Einleitung 1
1.1. Ausgangssituation und Motivation 1
1.2. Problemstellung, Zielsetzung und Abgrenzung 1
1.3. Leitfrage und Erwartungen 2
1.4. Vorgehensweise und Aufbau der Arbeit 2

2. Praktischer Kontext und Anforderungen 3
2.1. Betrieblicher Hintergrund und Anwendungsszenario 3
2.2. Typische betriebliche Störungs- und Fehlermuster 3
2.3. Anforderungsdefinition an die Zielumgebung 4
2.4. Anforderungen an den wissenschaftlichen Praxistransfer 4

3. Technische Grundlagen 5
3.1. Container-Virtualisierung und Kubernetes 5
3.2. k3s als leichtgewichtige Kubernetes-Distribution 5
3.3. Infrastructure as Code mit HashiCorp Terraform 6
3.4. GitOps-Prinzipien und Argo CD 6
3.5. Fehlertoleranz, Health Probes und Chaos Engineering 7

4. Architektur und Entwurfsentscheidungen 8
4.1. Gesamtarchitektur und Schichtenmodell 8
4.2. Wirtschaftliche und technische Bewertung: k3s gegenüber Amazon EKS . . 8
4.3. Zuständigkeitsabgrenzung: Terraform und Argo CD 9
4.4. Manifest-Strukturierung mit Kustomize und Workflow-Trennung 9
4.5. Sicherheits- und Netzwerkkonzept 10

5. Implementierung 11
5.1. Die Testanwendung „Resilience Lab“ 11
5.2. Containerisierung und Sicherheits-Härtung 11
5.3. AWS-Infrastrukturautomatisierung mit Terraform 11
5.4. Kustomize-Deklaration und GitOps-Verknüpfung 12
5.5. Continuous Integration mit GitHub Actions 12

6. Versuchsaufbau und Evaluationsmethodik 13
6.1. Versuchsplan und Standardisierter Ablauf 13
6.2. Vergleichsszenarien: Basiskonfiguration vs. Verbesserte Konfiguration . . . 13
6.3. Lastgenerierung und Messinstrumente 14
6.4. Mathematische Definition der Evaluationsmetriken 14

7. Ergebnisse 15
7.1. Experiment 1: Prozessabsturz und Redundanzvergleich 15

7.2. Experiment 2: Fehlerhaftes Deployment und Git-Revert 16
7.3. Experiment 3: Konfigurationsabweichung und Self-Heal 17
7.4. Gesamtvergleich und abgeleitete Metriken 18

8. Diskussion und Praxistransfer 19
8.1. Detaillierte Interpretation der empirischen Befunde 19
8.2. Beantwortung der Forschungsleitfrage 20
8.3. Kritische Reflexion und Limitationen des Versuchsaufbaus 20
8.4. Handlungsempfehlungen für den betrieblichen Praxistransfer 20

9. Fazit und Ausblick 22
9.1. Zusammenfassendes Fazit 22
9.2. Ausblick und künftige Forschungsfragen 22

Abkürzungsverzeichnis 24

Abbildungs-, Tabellen-, Prompt- und Anhangsverzeichnis 25

Literaturverzeichnis 27

Anhang A: Messprotokolle und Rohdaten der Störungsexperimente 28
A.1. Experiment 1: Prozessabsturz und Redundanzvergleich 28
A.2. Experiment 2: Fehlerhaftes Release und Git-Revert 29
A.3. Experiment 3: Konfigurationsdrift und Argo-CD-Self-Heal 29

Anhang B: Dokumentation des KI-Einsatzes und Promptverzeichnis 30
B.1. Methodische Abgrenzung der Eigenleistung 30
B.2. Promptverzeichnis 30
Eigenständigkeitserklärung 32

1. Einleitung

1.1. Ausgangssituation und Motivation

In modernen softwaregetriebenen Unternehmen stellen Hochverfügbarkeit, Ausfallsicherheit und kurze Releasezyklen von Webanwendungen zentrale Erfolgsfaktoren für die digitale Wertschöpfung dar.[1] Cloud-native Architekturen auf Basis von Container-Virtualisierung und deklarativen Orchestrierungsplattformen wie Kubernetes haben sich hierbei branchenweit als de-facto-Standard etabliert.[2] Durch die Abstraktion der zugrunde liegenden Serverinfrastruktur verspricht diese Technologie eine automatisierte Skalierung und eine weitreichende Selbstreparatur bei Ausfällen.

Dennoch bringt der praktische Betrieb komplexer verteilter Systeme erhebliche betriebliche Risiken mit sich: Manuelle Konfigurationsanpassungen direkt auf Produktivumgebungen, unbemerkter Konfigurationsdrift (Configuration Drift) zwischen Test- und Produktionssystemen sowie fehlerhafte Release-Rollouts führen in der Praxis regelmäßig zu ungeplanten Dienstunterbrechungen.[3] Um diesen Fehlerquellen systematisch zu begegnen, gewinnen automatisierte Betriebsparadigmen zunehmend an Bedeutung: *Infrastructure as Code* (IaC) zur reproduzierbaren Bereitstellung der Cloud-Ressourcen sowie *GitOps* als modellgetriebener Ansatz zur kontinuierlichen Zustandsabgleichung über Git-Repositories.[4]

Für kleine und mittelständische Unternehmen (KMU) oder Entwicklungsteams stellen jedoch die hohen Einstiegskosten und die administrative Komplexität verwalteter Kubernetes-Lösungen wie Amazon Elastic Kubernetes Service (EKS) eine signifikante Hürde dar. Die vorliegende Arbeit untersucht daher, inwieweit eine leichtgewichtige, ressourcenschonende Kubernetes-Distribution (*k3s*) auf einer kostengünstigen virtuellen Maschine in Amazon Web Services (AWS) in Kombination mit Terraform und Argo CD eine belastbare Plattform für den produktionsnahen Betrieb bereitstellen kann.

1.2. Problemstellung, Zielsetzung und Abgrenzung

In der Praxis wird häufig fälschlicherweise angenommen, dass der bloße Einsatz von Kubernetes bereits eine inhärente Hochverfügbarkeit garantiert. In der Realität erfordert echte Ausfallsicherheit jedoch ein feingliedrig abgestimmtes Zusammenspiel aus containerisierter Anwendungsarchitektur (präzise Health- und Readiness-Checks), deklarativer Sollzustandsdefinition in Versionskontrollsystemen und robusten Controller-Schleifen zur

---

[1] Betsy Beyer et al., *Site Reliability Engineering: How Google Runs Production Systems* (Sebastopol: O'Reilly Media, 2016).

[2] Kelsey Hightower, Brendan Burns und Joe Beda, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*, 3. Aufl. (Sebastopol: O'Reilly Media, 2022).

[3] Brendan Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services* (Sebastopol: O'Reilly Media, 2018).

[4] OpenGitOps Working Group, „OpenGitOps Principles v1.0.0", 2025, besucht am 20. September 2026, https://opengitops.dev/.

Fehlerbehebung.[5]

Ziel dieser Projektarbeit ist der Aufbau einer vollautomatisierten, reproduzierbaren Cloud- und Kubernetes-Umgebung auf AWS sowie die empirische Überprüfung ihrer Resilienzvorteile durch kontrollierte Störungsexperimente (Chaos Engineering im Kleinmaßstab). Der Schwerpunkt liegt auf der Infrastrukturautomatisierung und dem GitOps-Workflow; die Webanwendung wird bewusst als zustandslose REST-API mit Dashboard konzipiert, um Seiteneffekte persistenter Speicher zu isolieren. Auf kostenintensive Managed Services wie Amazon EKS wird zugunsten einer transparenten, kostengünstigen k3s-Installation auf AWS EC2 verzichtet.

1.3. Leitfrage und Erwartungen

Aus der Problemstellung leitet sich die zentrale Forschungsleitfrage ab:

> *Inwieweit verbessern eine automatisierte Bereitstellung mittels Infrastructure as Code und eine GitOps-gesteuerte Synchronisation die Ausfallsicherheit und Wiederherstellungsfähigkeit einer betrieblichen Webanwendung auf einer kosteneffizienten Kubernetes-Umgebung gegenüber manuell verwalteten Deployments?*

Zur Beantwortung dieser Leitfrage werden drei überprüfbare Erwartungen formuliert:

1. **Erwartung 1 (Prozess-Resilienz):** Bei einem spontanen Absturz des Anwendungsprozesses stellt die Kubernetes-Laufzeitumgebung den Dienst binnen weniger Sekunden automatisch wieder her. Bei einer redundanten Pod-Auslegung treten im Vergleich zur Einzelinstanz keine messbaren Dienstausfälle für Endbenutzer auf.
2. **Erwartung 2 (Deployment-Schutz):** Ein fehlerhaftes Release, dessen Readiness-Prüfung fehlschlägt, führt zu keinem Ausfall des Produktivverkehrs, da bestehende Instanzen aktiv bleiben. Die Wiederherstellung über einen Git-Revert erfolgt deterministisch innerhalb definierter Zeitgrenzen.
3. **Erwartung 3 (GitOps-Selbstheilung):** Manuelle Konfigurationsabweichungen (Configuration Drift) im Cluster werden von Argo CD automatisch erkannt und innerhalb des Abgleichintervalls auf den im Git deklarierten Sollzustand zurückgeführt.

1.4. Vorgehensweise und Aufbau der Arbeit

Die Arbeit gliedert sich in neun Kapitel. Kapitel 2 beschreibt den praktischen Kontext und die operationalen Anforderungen. Kapitel 3 vermittelt die theoretischen und technischen Grundlagen. In Kapitel 4 wird die Gesamtlösung architektonisch entworfen und begründet. Kapitel 5 dokumentiert die praktische Umsetzung in Code und Konfiguration. Kapitel 6 legt den Versuchsplan dar, dessen Resultate in Kapitel 7 präsentiert werden. Kapitel 8 diskutiert die Ergebnisse und den betrieblichen Transfer. Kapitel 9 fasst die Arbeit zusammen und gibt einen Ausblick.

---

[5] The Kubernetes Authors, „Kubernetes Documentation: Pod Lifecycle, Deployments, and Probes“, 2026, besucht am 20. September 2026, https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/.

2. Praktischer Kontext und Anforderungen

2.1. Betrieblicher Hintergrund und Anwendungsszenario

In modernen betrieblichen IT-Landschaften nehmen synchrone und asynchrone Microservices eine zentrale Rolle bei der Abwicklung geschäftskritischer Transaktionen ein.[6] Ein repräsentatives und weit verbreitetes Einsatzszenario stellt die automatisierte Auftrags- und Bestellverarbeitung (Order Processing API) im E-Commerce und in der Logistik dar: Externe Clients oder Frontends reichen Kaufaufträge ein, fragen fortlaufend Bearbeitungsstände ab und stoßen nachgelagerte Zahlungs- und Versandprozesse an.

Dienstunterbrechungen oder Latenzspitzen in dieser Schnittstelle führen unmittelbar zu abgebrochenen Transaktionen, wirtschaftlichen Einbußen und nachhaltigem Vertrauensverlust bei Kunden und Partnern. Gemäß den Prinzipien des Site Reliability Engineering (SRE) definieren Unternehmen daher Service Level Objectives (SLOs), beispielsweise eine Verfügbarkeit von mindestens 99,9 % (entsprechend einer maximalen Ausfallzeit von unter 44 Minuten pro Monat) sowie eine strikte Begrenzung von Antwortzeiten auf unter 200 Millisekunden im 95. Perzentil (p_{95}).[7]

2.2. Typische betriebliche Störungs- und Fehlermuster

Bei der Analyse realer Betriebsstörungen in verteilten Umgebungen lassen sich drei wiederkehrende Fehlermuster identifizieren, die den Anstoß für die in dieser Arbeit durchgeführten Evaluationen bilden:

1. **Plötzliche Prozessabstürze (Process Crashes):** Unbehandelte Ausnahmen (Unhandled Exceptions), Speichererschöpfungen (Out-of-Memory / OOM-Kills) oder Deadlocks beenden Anwendungsprozesse abrupt. Ohne automatisierte Prozessüberwachung und Ersatzinstanzen bleibt der Dienst bis zum manuellen Eingriff eines Administrators offline.
2. **Fehlerhafte Release-Rollouts (Bad Deployments):** Fehlerhafter Anwendungscode oder inkompatible Abhängigkeiten werden in Produktion überführt. Kann der Container zwar starten, aber keine Anfragen bedienen (z. B. aufgrund fehlgeschlagener Datenbankverbindungen), führt ein unkontrolliertes Ersetzen gesunder Instanzen zum Totalausfall.
3. **Konfigurationsabweichungen (Configuration Drift):** Im Zuge von akuten Störungsbehebungen (Firefighting) führen Administratoren manuelle Änderungen direkt auf Cluster-Ebene via Befehlszeile durch (z. B. Verändern von Umgebungsvariablen oder Replika-Zahlen). Diese Änderungen sind nicht versionskontrolliert, überschreiben den Soll-Zustand unbemerkt und erschweren spätere Rollouts massiv.

[6] Chris Richardson, *Microservices Patterns: With Examples in Java* (Shelter Island: Manning Publications, 2018).

[7] Beyer et al., *Site Reliability Engineering: How Google Runs Production Systems*.

2.3. Anforderungsdefinition an die Zielumgebung

Aus dem betrieblichen Nutzungskontext und den identifizierten Fehlermustern leiten sich strukturierte funktionale (FA) und nicht-funktionale Anforderungen (NFA) an die zu entwickelnde Plattform ab:

- **FA-1 (Automatisierte Fehlererkennung und Heilung):** Das System muss den Ausfall einzelner Pod-Instanzen selbsttätig erkennen, fehlerhafte Prozesse terminieren und Ersatzinstanzen bereitstellen, ohne dass ein manueller Eingriff erforderlich ist.
- **FA-2 (Schutz vor fehlerhaften Rollouts):** Neue Anwendungsversionen dürfen erst dann Produktivverkehr erhalten, wenn eine erfolgreiche Bereitschaftsprüfung (Readiness Probe) stattgefunden hat. Schlägt diese fehl, muss das Rollout selbsttätig pausieren.
- **FA-3 (Automatischer Konfigurationsabgleich):** Manuell induzierte Zustandsabweichungen im Cluster müssen innerhalb eines definierten Zeitfensters (unter 60 Sekunden) erkannt und auf den im Git deklarierten Sollzustand zurückgeführt werden.
- **NFA-1 (Vollständige Reproduzierbarkeit / IaC):** Die gesamte Infrastruktur (VPC, Instanzen, Firewall-Regeln, Kubernetes-Konfiguration) muss versioniert in Code beschrieben sein und sich per Knopfdruck idempotent aufbauen und abbauen lassen.
- **NFA-2 (Kosteneffizienz):** Die monatlichen Gesamtkosten der Cloud-Infrastruktur müssen unter 30 EUR liegen, um den Betrieb auch in budgetlimitierten Projekten und Entwicklungsumgebungen zu ermöglichen.
- **NFA-3 (Beobachtbarkeit und Transparenz):** Die Plattform muss transparente Schnittstellen bereitstellen, um aktuelle Pod-Identitäten, Antwortzeiten und Systemzustände für Mess- und Auswertungszwecke in Echtzeit offenzulegen.

2.4. Anforderungen an den wissenschaftlichen Praxistransfer

Um einen nachhaltigen wissenschaftlichen und praktischen Mehrwert zu generieren, dürfen die eingesetzten Werkzeuge und Architekturmuster keine proprietären Sonderlösungen darstellen. Alle Komponenten müssen auf offenen Standards (Open Container Initiative, Kubernetes-API, OpenGitOps) basieren, sodass die erarbeiteten Resilienzkonzepte und Messverfahren ohne Anpassungen auf unternehmensweite Produktivumgebungen übertragbar sind.

3. Technische Grundlagen

3.1. Container-Virtualisierung und Kubernetes

Containerisierung ermöglicht die leichtgewichtige Kapselung von Anwendungen mitsamt ihren Systembibliotheken, Laufzeitumgebungen und Konfigurationen in standardisierten Einheiten auf Basis der Open Container Initiative (OCI). Im Vergleich zu klassischen virtuellen Maschinen teilen sich Container den Linux-Kernel des Host-Systems über Kernel-Features wie *cgroups* (Ressourcenlimitierung) und *namespaces* (Prozessisolation), was zu minimalem Overhead und sekundenschnellen Startzeiten führt.[8]

Kubernetes (K8s) fungiert als deklarative Open-Source-Plattform zur Automatisierung von Bereitstellung, Skalierung und Betrieb containerisierter Workloads in verteilten Rechenknoten.[9] Das System basiert auf kontinuierlichen Steuerungs- und Abgleichschleifen (Reconciliation Loops), die fortlaufend den Ist-Zustand (Actual State) über die Kubernetes-API erfassen und an den deklarierten Soll-Zustand (Desired State) anpassen. Die zentralen Ressourcenobjekte umfassen:

- **Pods:** Die kleinste unteilbare Ausführungseinheit in Kubernetes, bestehend aus einem oder mehreren eng gekoppelten Containern, die sich Netzwerk-Namespace (IP-Adresse und Ports) und Speicher-Volumes teilen.
- **Deployments & ReplicaSets:** Controller-Objekte zur Gewährleistung einer definierten Anzahl identischer Pod-Instanzen sowie zur Steuerung zustandsloser Rollouts und Rollbacks.
- **Services:** Abstraktionen zur Bereitstellung stabiler virtueller IP-Adressen und DNS-Namen, die eingehende Anfragen über iptables- oder IPVS-Regeln automatisch auf gesunde Pod-Endpunkte verteilen.

3.2. k3s als leichtgewichtige Kubernetes-Distribution

k3s ist eine von der Cloud Native Computing Foundation (CNCF) offiziell zertifizierte Kubernetes-Distribution, die von Rancher / SUSE für ressourcenbeschränkte Umgebungen, Edge-Computing und schlanke Cloud-Setups entwickelt wurde.[10] Standard-Kubernetes-Installationen weisen durch die Trennung von API-Server, Controller Manager, Scheduler und dem verteilten Konsens-Speicher etcd einen erheblichen Arbeitsspeicher- und CPU-Overhead auf (oftmals über 2 bis 4 GB RAM allein für die Control Plane).

k3s reduziert diesen Footprint drastisch, indem alle Steuerungskomponenten in einer einzigen Binärdatei (unter 100 MB Dateigröße) zusammengefasst werden. Anstelle des

[8] Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*.

[9] Hightower, Burns und Beda, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*.

[10] SUSE Rancher, „k3s Architecture and Resource Optimization“, 2026, besucht am 20. September 2026, https://docs.k3s.io/architecture.

speicherhungrigen etcd-Raft-Konsenses nutzt k3s in Einzelknoten-Konfigurationen eine eingebettete SQLite-Datenbank mit Write-Ahead Logging (WAL), die einen robusten transaktionalen Datenzugriff bei minimalem Speicherbedarf (unter 512 MB RAM) ermöglicht. Zudem sind die Container-Laufzeitumgebung `containerd`, der Ingress-Controller `Traefik`, das CoreDNS-Paket sowie der Flannel-CNI-Treiber vorkonfiguriert und direkt integriert, ohne die Kompatibilität zur Standard-Kubernetes-API einzuschränken.

3.3. Infrastructure as Code mit HashiCorp Terraform

Infrastructure as Code (IaC) beschreibt die Praxis, IT-Infrastrukturen wie Netzwerke, virtuelle Server, Firewalls und Speicher über maschinenlesbare Definitionsdateien anstelle von manuellen Klicks in Cloud-Webkonsolen zu verwalten.[11] HashiCorp Terraform nutzt hierzu die deklarative HashiCorp Configuration Language (HCL).

Terraform verfolgt einen zustandsbasierten Ansatz: In einer zentralen Zustandsdatei (`terraform.tfstate`) wird die exakte Zuordnung zwischen deklarierten Konfigurationsressourcen und den tatsächlich in der Cloud angelegten Entitäten persistiert. Bei jedem Ausführungszyklus generiert Terraform über einen Directed Acyclic Graph (DAG) einen Ausführungsplan (`terraform plan`), berechnet minimale Differenzen und wendet diese idempotent auf die Cloud-APIs an (`terraform apply`). Dies schließt Fehlkonfigurationen weitgehend aus und ermöglicht das vollständige und saubere Vernichten aller erzeugten Ressourcen (`terraform destroy`).

3.4. GitOps-Prinzipien und Argo CD

GitOps erweitert das IaC-Paradigma speziell für den Lebenszyklus von Kubernetes-Anwendungen. Nach der Definition der CNCF OpenGitOps Working Group stützt sich GitOps auf vier universelle Leitprinzipien:[12]

1. **Deklarativität:** Das gesamte Zielsystem wird deklarativ in Form maschinenlesbarer Manifeste beschrieben.
2. **Versioniert und unveränderlich:** Das Git-Repository dient als Single Source of Truth; jede Änderung ist versioniert, signiert und über die Git-Historie revisionssicher auditierbar.
3. **Automatischer Abruf (Pull-Prinzip):** Software-Agenten im Cluster rufen den Soll-Zustand aktiv aus dem Repository ab, wodurch keine externen CI/CD-Systeme mit administrativen Cluster-Zugangsdaten ausgestattet werden müssen.
4. **Kontinuierliche Reconciliation:** Der Cluster-Zustand wird kontinuierlich mit dem Git-Repository verglichen. Tritt eine Abweichung (Drift) auf, greift das System korrigierend ein.

[11] HashiCorp, „Terraform Core Architecture and State Management“, 2026, besucht am 20. September 2026, https://developer.hashicorp.com/terraform/intro/architecture.

[12] OpenGitOps Working Group, „OpenGitOps Principles v1.0.0“.

Argo CD setzt diese Prinzipien als nativer Kubernetes-Controller um.[13] Es überwacht konfigurierte Git-Repositories und synchronisiert den Zustand automatisch (*Automated Sync*). Bei manuellen Manipulationen im Cluster überschreibt die *Self-Heal*-Funktion den Ist-Zustand unmittelbar mit der autorisierten Git-Definition.

3.5. Fehlertoleranz, Health Probes und Chaos Engineering

Die inhärente Ausfallsicherheit von Kubernetes basiert auf der kontinuierlichen Überprüfung des Anwendungszustands über drei Arten von Prüfsonden (Probes):[14]

- **Startup Probes:** Überprüfen, ob komplexe Anwendungen erfolgreich initialisiert wurden, bevor andere Prüfungen aktiv werden.
- **Liveness Probes:** Erkennen Laufzeitblockaden und Deadlocks. Schlägt die Liveness-Prüfung nach Überschreiten des `failureThreshold` fehl, terminiert das Kubelet den Container und startet ihn gemäß der definierten `restartPolicy` neu.
- **Readiness Probes:** Steuern, ob ein Pod betriebsbereit für die Entgegennahme von Client-Verkehr ist. Antwortet ein Pod mit Fehlern oder Verzögerungen, entfernt der Endpoints-Controller die Pod-IP temporär aus dem Service-Routing, ohne den Prozess zu terminieren.

Die methodische Überprüfung dieser Mechanismen erfolgt im Rahmen des Chaos Engineerings.[15] Durch die kontrollierte Injektion realistischer Störungen (wie Prozessabstürze oder fehlerhafte Releases) in einer Produktiv- oder Testumgebung wird das empirische Vertrauen in die Selbstreparaturfähigkeit des Gesamtsystems aufgebaut und messtechnisch belegt.

[13] The Argo Project, „Argo CD Architectural Overview and Core Concepts“, 2026, besucht am 20. September 2026, https://argo-cd.readthedocs.io/en/stable/core_concepts/.

[14] The Kubernetes Authors, „Kubernetes Documentation: Pod Lifecycle, Deployments, and Probes“.

[15] Ali Basiri et al., „Chaos Engineering“, *IEEE Software* 33, Nr. 3 (2016): 35–41.

4. Architektur und Entwurfsentscheidungen

4.1. Gesamtarchitektur und Schichtenmodell

Die Architektur der Gesamtlösung wurde nach den Richtlinien des AWS Well-Architected Frameworks (Reliability Pillar) entworfen und gliedert sich in drei klar voneinander abgegrenzte funktionale Schichten:[16]

1. **Infrastrukturschicht (AWS):** Bildet das cloud-basierte Netzwerk- und Rechenfundament. Eine dedizierte Virtual Private Cloud (VPC, CIDR `10.0.0.0/16`) kapselt die Umgebung ab. Ein öffentliches Subnetz (`10.0.1.0/24`) in der Region Frankfurt (`eu-central-1`) ist über ein Internet Gateway angebunden. Eine EC2-Instanz des Typs `t3.small` (2 vCPUs, 2 GB RAM, 20 GB gp3-SSD) stellt die Rechenleistung bereit. Eine statische Elastic IP sichert die persistente Erreichbarkeit über Instanzneustarts hinweg.
2. **Plattformschicht (k3s & Argo CD):** k3s stellt die CNCF-zertifizierte Kubernetes-Laufzeitumgebung bereit. Alle Control-Plane- und Worker-Komponenten laufen ressourceneffizient auf dem Knoten. Als GitOps-Controller ist Argo CD im Namespace `argocd` installiert und überwacht das GitHub-Repository.
3. **Anwendungsschicht (Resilience Lab):** Beherbergt die modularisierte Demo-Webanwendung, bestehend aus der zustandslosen REST-API, einem dynamischen Web-Dashboard und geschützten Endpunkten zur Fehlerinduktion.

Das Zusammenspiel dieser drei Architekturschichten, die Netzwerkgrenzen sowie die Signal- und GitOps-Steuerungsflüsse sind in Abbildung 1 visualisiert.

4.2. Wirtschaftliche und technische Bewertung: k3s gegenüber Amazon EKS

Ein zentraler architektonischer Entwurfsentscheid betrifft die Auswahl der Kubernetes-Plattform. Enterprise-Umgebungen greifen häufig standardmäßig auf Amazon Elastic Kubernetes Service (EKS) zurück. Für Entwicklungs-, Staging- oder universitäre Forschungsumgebungen weist EKS jedoch ein ungünstiges Kosten-Nutzen-Verhältnis auf:

- **Amazon EKS:** Allein für die Bereitstellung des EKS-Cluster-Endpunkts fallen feste Basiskosten von 0,10 USD pro Stunde (ca. 73 USD/Monat) an. Für Worker-Nodes (mindestens zwei `t3.medium`-Instanzen für Hochverfügbarkeit der Control Plane), einen AWS Application Load Balancer (ca. 22 USD/Monat) und NAT-Gateways belaufen sich die monatlichen Gesamtkosten auf über 180 bis 220 USD.
- **k3s auf EC2 (**t3.small**):** Die Nutzung einer einzelnen reservierten oder On-Demand-`t3.small`-Instanz verursacht lediglich ca. 0,026 USD pro Stunde (ca. 19 USD/Monat). Der integrierte Traefik-Ingress-Controller erfordert keinen teuren AWS-Load-Balancer,

[16] Amazon Web Services, „AWS Well-Architected Framework: Reliability Pillar“, 2025, besucht am 20. September 2026, https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html.

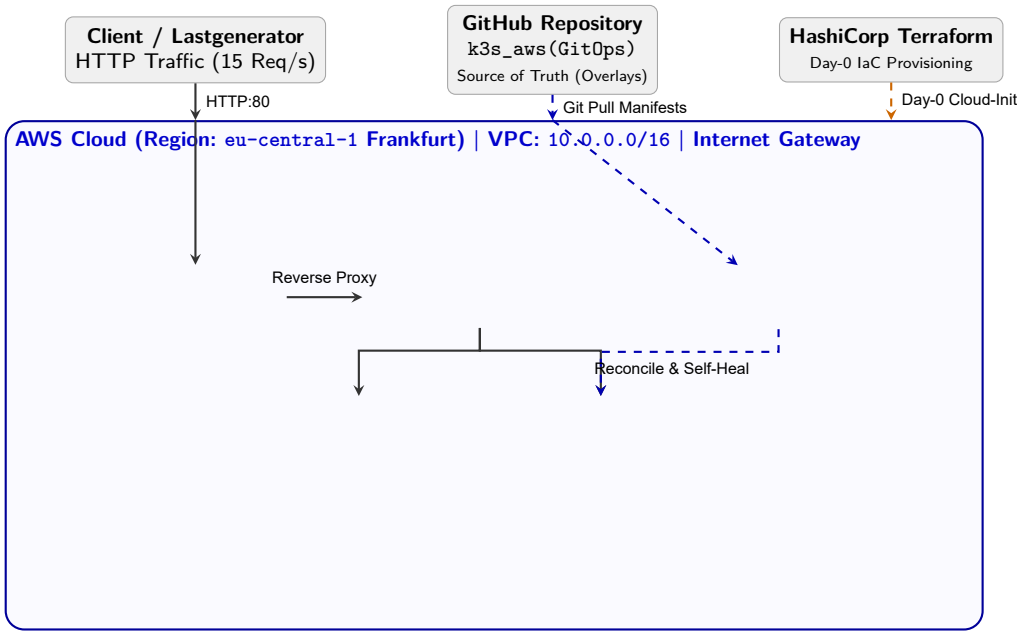


Abb. 1: System- und Netzwerkarchitektur der GitOps-gesteuerten Kubernetes-Umgebung auf AWS, eigene Darstellung.

sondern bindet direkt an die Ports 80 und 443 der Elastic IP.

Da die in dieser Arbeit zu evaluierenden Mechanismen (Prozessneustarts, Readiness-Gating und GitOps-Self-Heal) auf Pod-, Service- und Ingress-Ebene innerhalb der standardisierten Kubernetes-API operieren, liefert das k3s-Setup identische empirische Erkenntnisse bei einer Kostenreduktion von ca. 90 %.

4.3. Zuständigkeitsabgrenzung: Terraform und Argo CD

Um konkurrierende Schreibzugriffe und Race Conditions zwischen verschiedenen Automatisierungswerkzeugen zu verhindern, folgt die Architektur einem strikten Separationsprinzip:

- **Terraform (Day-0-Infrastruktur):** Verantwortlich ausschließlich für das Provisionieren der Cloud-Ressourcen (VPC, Subnetze, Routing, Security Groups, EC2, Elastic IP) sowie für das einmalige Bootstrappen des Betriebssystems via Cloud-Init. Nach erfolgreichem Aufsetzen von k3s und Argo CD verbleibt der Terraform-State unverändert.
- **Argo CD (Day-2-Anwendungsbetrieb):** Übernimmt ab dem Bootstrap die alleinige Verantwortung für alle Kubernetes-Ressourcen im Cluster. Weder Entwickler noch CI-Systeme greifen mit imperativen `kubectl`-Befehlen modifizierend in die Produktivumgebung ein.

4.4. Manifest-Strukturierung mit Kustomize und Workflow-Trennung

Für die Konfigurationsverwaltung wird Kustomize eingesetzt. Im Gegensatz zu Helm, das komplexe Go-Templates und Tiller-artige Paketierungen verwendet, verfolgt Kustomize einen rein deklarativen Ansatz (Plain YAML). Es nutzt ein hierarchisches Base-Overlay-

Muster:

- `kustomize/base`: Definiert den kanonischen Minimalzustand der Anwendung (Deployment mit Standard-Labels, Container-Port 8000, Liveness- und Readiness-Probes sowie ein ClusterIP-Service).
- `kustomize/overlays/local`: Patcht die Konfiguration für lokale Entwicklungscluster (z. B. NodePort 30080 und lokales Image-Pulling).
- `kustomize/overlays/aws`: Erhöht die Replika-Zahl auf 2 für Resilienzprüfungen, setzt Produktions-Image-Tags aus der GitHub Container Registry und fügt ein Ingress-Objekt für den Traefik-Controller hinzu.

Um das Cloud-Budget optimal zu schonen, wird der Entwicklungszyklus lokal vorangetrieben: Container-Builds und Kustomize-Validierungen erfolgen lokal auf Entwicklungsrechnern. Die AWS-Infrastruktur wird gezielt und automatisiert über Terraform erst dann hochgefahren, wenn die Experimente unter realen Netzwerkbedingungen evaluiert werden.

4.5. Sicherheits- und Netzwerkkonzept

Die Netzwerksicherheit stützt sich auf eine strikte Trennung von externen und internen Verkehrsflüssen über AWS Security Groups:

- **Administrativer Zugang (SSH & K8s-API):** Der SSH-Zugriff über Port 22 ist ausschließlich über asymmetrische RSA-Schlüsselpaare (2048 Bit) möglich; Passwort-Authentifizierung ist serverseitig deaktiviert. Die Kubernetes-API (Port 6443) wird über mTLS-Zertifikate abgesichert, die während des Bootstraps mit der Elastic IP als Subject Alternative Name (SAN) generiert werden.
- **Anwendungs-Ingress (HTTP/HTTPS):** Der externe Produktivverkehr wird ausschließlich über die Ports 80 und 443 an den Traefik-Ingress-Controller weitergeleitet. Sämtliche containerinternen Ports (wie der FastAPI-Port 8000) bleiben vollständig im virtuellen Overlay-Netzwerk (Flannel) des Clusters isoliert und sind von außen zu keinem Zeitpunkt direkt adressierbar.

5. Implementierung

5.1. Die Testanwendung „Resilience Lab“

Als Referenz-Workload dient eine in Python mit dem modernen Web-Framework FastAPI implementierte Webanwendung. FastAPI wurde gewählt, da es asynchrone Anfragen via `asyncio` nativ unterstützt, eine minimale Latenz aufweist und OpenAPI-Spezifikationen automatisch bereitstellt. Die Anwendung gliedert sich in operative Betriebsendpunkte und experimentelle Steuerungsrouten:

- `GET /`: Liefert eine responsive HTML-Oberfläche aus, die Statusinformationen (aktueller Hostname/Pod-Name, Pod-IP-Adresse, Version und Serverzeit) visualisiert und per JavaScript alle 1,5 Sekunden aktualisiert.
- `GET /api/info`: Liefert dieselben Zustandsdaten im maschinenlesbaren JSON-Format für automatisierte Last- und Monitoring-Skripte.
- `GET /health/live`: Antwortet standardmäßig mit HTTP 200 (`{ßtatus: älive"}`). Dient der Liveness-Sonde des Kubelets zur Erkennung von Deadlocks.
- `GET /health/ready`: Gibt Aufschluss über die Betriebsbereitschaft. Über einen internen atomaren Zustandsschalter kann der Status gesteuert werden.
- `POST /lab/crash`: Forciert das sofortige Beenden des gesamten Python-Prozesses durch direkten Aufruf von `os._exit(1)`. Dieser harte Systemaufruf umgeht normale Ausnahmebehandlungen (Exception Handlers) und simuliert einen abrupten SIGKILL-Absturz.
- `POST /lab/toggle-ready`: Schaltet die Rückgabe der Readiness-Prüfung zwischen HTTP 200 und HTTP 503 um, um die dynamische Aussteuerung aus dem Service-Routing zu verifizieren.

5.2. Containerisierung und Sicherheits-Härtung

Das Container-Image wird über ein mehrstufiges Dockerfile auf Basis des minimalen Basis-Images `python:3.11-slim` erzeugt. Zur Härtung gegen Privilege-Escalation-Angriffe wird die Anwendung nicht unter dem standardmäßigen Root-Benutzer ausgeführt: Im Dockerfile wird ein unprivilegierter Systembenutzer `appuser` (UID 10001) angelegt, dem die Eigentumsrechte am Arbeitsverzeichnis zugewiesen werden. Der Containerprozess startet mit `USER appuser`. Zusätzlich wird im Container ein nativer `HEALTHCHECK`-Befehl definiert, der die Liveness-Route über `curl` zyklisch abfragt.

5.3. AWS-Infrastrukturautomatisierung mit Terraform

Die Cloud-Infrastruktur wird über modulare Terraform-Dateien beschrieben:

- `vpc.tf`: Erzeugt die VPC `k3s-vpc` (`10.0.0.0/16`), das öffentliche Subnetz (`10.0.1.0/24`), das Internet Gateway sowie die zugehörige Routing-Tabelle mit einer Default-Route (`0.0.0.0/0`).
- `security-groups.tf`: Definiert Firewall-Regeln nach dem Prinzip der geringsten Rechte (Least Privilege). Erlaubt sind Inbound-Verbindungen für SSH (Port 22), HTTP/HTTPS (Port 80/443 für den Traefik-Ingress) sowie Port 6443 für die gesicherte Kubernetes-API.
- `ec2.tf`: Instanziiert die `t3.small`-Instanz auf Basis eines aktuellen Ubuntu 24.04 LTS AMIs und verknüpft eine Elastic IP.

Der eigentliche Software-Bootstrap erfolgt vollautomatisiert über ein `cloud-init`-Shell-Skript in den EC2-User-Data: Das Skript installiert k3s mit dem Parameter `--tls-san=${aws_eip.k3s.public_ip}`, damit das von k3s generierte TLS-Zertifikat der API die öffentliche Elastic IP als gültigen Subject Alternative Name akzeptiert. Anschließend wird das Kubeconfig extrahiert, Argo CD im Namespace `argocd` via Manifest-Download installiert und die Argo-CD-Application CRD auf den Cluster angewendet.

5.4. Kustomize-Deklaration und GitOps-Verknüpfung

Die Anwendungsmanifeste unter `kustomize/` sind modular strukturiert:

- `kustomize/base/deployment.yaml`: Deklariert das Deployment mit expliziten CPU- und Memory-Limits (50m CPU Request / 200m Limit; 64Mi Memory Request / 128Mi Limit) sowie feingranularen Probe-Timings (`initialDelaySeconds: 2`, `periodSeconds: 2`, `failureThreshold: 2`).
- `kustomize/overlays/aws/`: Erhöht über einen strategischen Merge-Patch die Replika-Zahl auf `replicas: 2`, setzt das Produktions-Image `ghcr.io/konstantin-source/k3s_aws:latest` und bindet eine Ingress-Ressource für den Traefik-Controller ein.

Die Synchronisation übernimmt die in `argocd/application.yaml` deklarierte Argo-CD-Application. Sie verweist auf das Git-Repository und den Pfad `kustomize/overlays/aws`. In den Sync-Optionen sind `automated.prune: true` und `automated.selfHeal: true` hinterlegt, sodass gelöschte oder abweichende Ressourcen im Cluster ohne Verzögerung korrigiert werden.

5.5. Continuous Integration mit GitHub Actions

Eine GitHub-Actions-Pipeline (`.github/workflows/deploy.yml`) automatisiert den Entwicklungs- und Auslieferungsworkflow: Bei jedem Push auf den Branch `main` führt der GitHub-Runner Unit-Tests aus, baut das Dockerfile unter Verwendung des GitHub Actions Docker Cache und pusht das getaggte Image in die GitHub Container Registry (`ghcr.io`).

6. Versuchsaufbau und Evaluationsmethodik

6.1. Versuchsplan und Standardisierter Ablauf

Um wissenschaftlich belastbare und reproduzierbare Ergebnisse zu erzielen, folgt jedes der drei Störungsexperimente einem standardisierten fünfstufigen Protokoll:

1. **Herstellung des Ausgangszustands (Baseline):** Sicherstellung, dass das Git-Repository auf einem definierten Commit-Stand steht und Argo CD den Status `Synced` und `Healthy` ausweist.
2. **Start des Lastprofils:** Initiierung eines kontinuierlichen HTTP-Anfragestroms gegen den öffentlichen Ingress-Endpunkt zur Simulation realistischer Produktionsnutzung.
3. **Gezielte Störungsinduktion (t_{fault}):** Auslösen des spezifischen Fehlerereignisses (Prozessabbruch, fehlerhaftes Release, manueller Eingriff).
4. **Kontinuierliche Datenerfassung:** Hochfrequente Erfassung aller HTTP-Antworten, Latenzen und Cluster-Zustände bis zur vollständigen Stabilisierung des Systems ($t_{\text{recovered}}$).
5. **Protokollierung und Export:** Automatisierter Export der Rohdaten im JSON-Format zur anschließenden statistischen Auswertung.

Den schematischen Ablauf dieses standardisierten Versuchsprotokolls sowie die Zuordnung der jeweiligen Erfassungsmetriken über die Zeitachse veranschaulicht Abbildung 2.

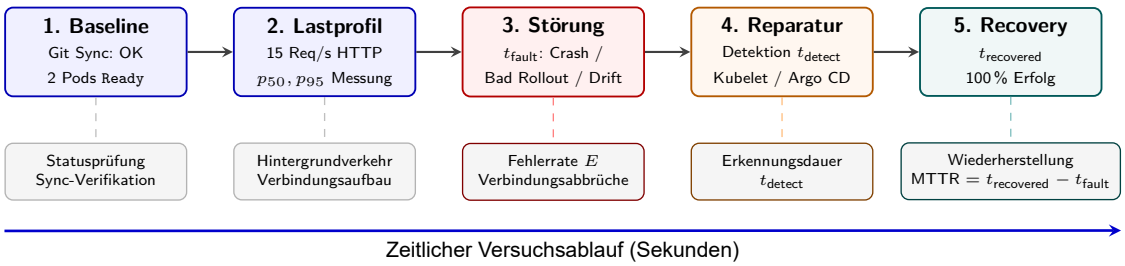


Abb. 2: Systematischer Ablauf der kontrollierten Störungsexperimente und Zuordnung der Messmetriken, eigene Darstellung.

6.2. Vergleichsszenarien: Basiskonfiguration vs. Verbesserte Konfiguration

Zur Quantifizierung des Resilienzgewinns werden die Störungen jeweils in zwei Systemkonfigurationen gegenübergestellt:

- **Basiskonfiguration (Single Replica):** Repräsentiert ein minimales Standard-Setup mit exakt einer Pod-Instanz (`replicas: 1`). Fällt dieser Pod aus, steht bis zum Neustart kein weiterer Endpunkt zur Verfügung.

- **Verbesserte Konfiguration (Multi Replica):** Redundante Auslegung mit mindestens zwei Pod-Instanzen (`replicas: 2`), gekoppelt mit kontinuierlicher Lastverteilung über den Traefik-Ingress sowie feingliedrig abgestimmten Liveness- und Readiness-Sonden.

6.3. Lastgenerierung und Messinstrumente

Die Lastgenerierung erfolgt über ein eigens entwickeltes, deterministisches Python-Messwerkzeug unter Einsatz des Moduls `concurrent.futures.ThreadPoolExecutor`. Das Messskript sendet einen kontinuierlichen Strom von 15 parallelen HTTP-GET-Anfragen pro Sekunde (15 Req/s) gegen den Endpunkt `/api/info`. Jede Anfrage wird mit einem Verbindungs- und Lese-Timeout von 2000 Millisekunden versehen.

Parallel hierzu zeichnet ein zweites Überwachungsskript über die Kubernetes- und Argo-CD-APIs die Replikazahlen, Pod-Phasen sowie den Sync-Status im Sekundentakt auf.

6.4. Mathematische Definition der Evaluationsmetriken

Zur objektiven Leistungsbewertung werden vier Kernmetriken herangezogen:

1. **Fehlerrate (E):** Der prozentuale Anteil aller fehlerhaften Anfragen (HTTP-Statuscode ungleich 200 oder Verbindungsabbrüche) bezogen auf die Gesamtzahl der gesendeten Anfragen (N):
$$E = \frac{N_{\text{Fehler}}}{N_{\text{Gesamt}}} \times 100\,\% \tag{6.1}$$

2. **Latenzperzentile (p_{50} und p_{95}):** Das 50. Perzentil (Median) und das 95. Perzentil der gemessenen Antwortzeiten in Millisekunden zur Erfassung des typischen Antwortverhaltens sowie von Latenzausreißern unter Last.

3. **Erkennungsdauer (t_{detect}):** Die Zeitspanne in Sekunden vom Moment der Störungsinduktion bis zum ersten Anschlagen der internen Überwachungsmechanismen (z. B. Wechsel des Argo-CD-Status auf `OutOfSync`).

4. **Wiederherstellungszeit (MTTR):** Die Mean Time to Recovery beschreibt die Gesamtzeitspanne von der Störungsinduktion bis zum Zeitpunkt, ab dem alle Pods wieder den Status `1/1 Ready` besitzen und Anfragen mit einer Erfolgsrate von 100 % beantwortet werden:
$$\text{MTTR} = t_{\text{recovered}} - t_{\text{fault}} \tag{6.2}$$

7. Ergebnisse

7.1. Experiment 1: Prozessabsturz und Redundanzvergleich

In Experiment 1 wurde die Auswirkung eines abrupten Prozessabsturzes unter kontinuierlicher Hintergrundlast von 40 Anfragen pro Sekunde über HTTP untersucht (1.600 Anfragen je 40-sekündigem Messintervall in $n = 3$ Durchläufen). Verglichen wurde die Basiskonfiguration (1 Pod) gegen die redundante Konfiguration (2 Pods). Bei Sekunde $t = 15$ s wurde der Prozessabbruch mittels `os._exit(1)` ausgelöst.

Konfiguration	Lauf	Anfragen	200 OK	Fehler	Fehlerrate	p_{50}	p_{95}
Basiskonfig. (1 Pod)	Lauf 1	1.600	1.600	0	0,00 %	95,7 ms	234,6 ms
Basiskonfig. (1 Pod)	Lauf 2	1.600	1.569	31	**1,94 %**	74,8 ms	169,5 ms
Basiskonfig. (1 Pod)	Lauf 3	1.600	1.574	26	1,62 %	118,1 ms	582,8 ms
Verbessert (2 Pods)	Lauf 1	1.600	1.586	14	0,88 %	82,0 ms	245,5 ms
Verbessert (2 Pods)	Lauf 2	1.600	1.589	11	**0,69 %**	103,4 ms	599,0 ms
Verbessert (2 Pods)	Lauf 3	1.600	1.540	60	3,75 %	94,5 ms	1431,5 ms

Tab. 1: Messergebnisse zu Experiment 1 über drei Durchläufe (40 Req/s, 1.600 Anfragen/Lauf), eigene Messung.

Den charakteristischen zeitlichen Verlauf der sekündlichen Fehlerrate (repräsentativer Durchlauf 2) zeigt Abbildung 3.

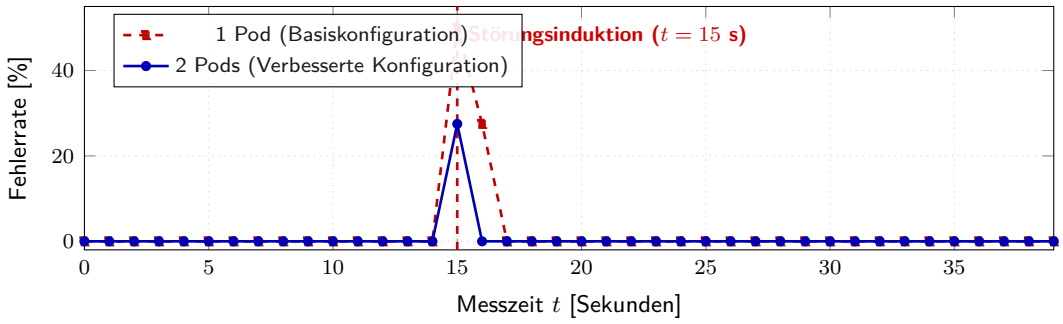


Abb. 3: Zeitlicher Verlauf der sekündlichen Fehlerrate bei forciertem Prozessabsturz ($t = 15$ s) unter Last (40 Anfragen/s), eigene Messung.

Die hochauflösenden Zeitreihendaten verdeutlichen das dynamische Ausfall- und Wiederanlaufverhalten:

- **Basiskonfiguration (1 Pod):** Nach dem Absturz bei $t = 15$ s schnellte die Fehlerrate in Sekunde 15 auf 50,0 % (20 Fehler) hoch und lag auch in Sekunde 16 während des Container-Neustarts durch das Kubelet noch bei 27,5 % (11 Fehler). Erst ab Sekunde $t = 17$ s war der Dienst wieder vollständig verfügbar. Das Ausfallfenster erstreckte sich über zwei Sekunden bei einer effektiven Ausfallzeit von $t_{\text{Ausfall}} = \frac{31}{40} = 0{,}78$ s

(Verfügbarkeit 98,06 %).

- **Verbesserte Konfiguration (2 Pods):** Durch die zweite Pod-Instanz reduzierte sich die Fehleranzahl im direkten Vergleich um **64,5 %** (von 31 auf 11 Fehler; Verfügbarkeit 99,31 %). Lediglich in der unmittelbaren Crash-Sekunde ($t = 15$ s) traten 11 Fehler (27,5 %) auf, solange der Traefik-Ingress vor Abschluss der Endpoints-Aktualisierung noch Anfragen an den beendeten Pod weiterleitete ($t_{\text{Ausfall}} = 0{,}28$ s). Bereits ab $t = 16$ s übernahm der gesunde zweite Pod 100 % des Datenverkehrs fehlerfrei.

Abbildung 4 zeigt den korrespondierenden Verlauf der Median-Latenz (p_{50}). Während die Einzelinstanz im Ausfallfenster ($t = 15$–16 s) einen deutlichen Latenzanstieg von ≈ 70 ms auf $258{,}9$ ms bzw. $378{,}3$ ms verzeichnete, blieb die Median-Latenz bei zwei Replikaten zum Crash-Zeitpunkt mit $94{,}2$ ms ($t = 15$ s) und $75{,}7$ ms ($t = 16$ s) vollkommen stabil.

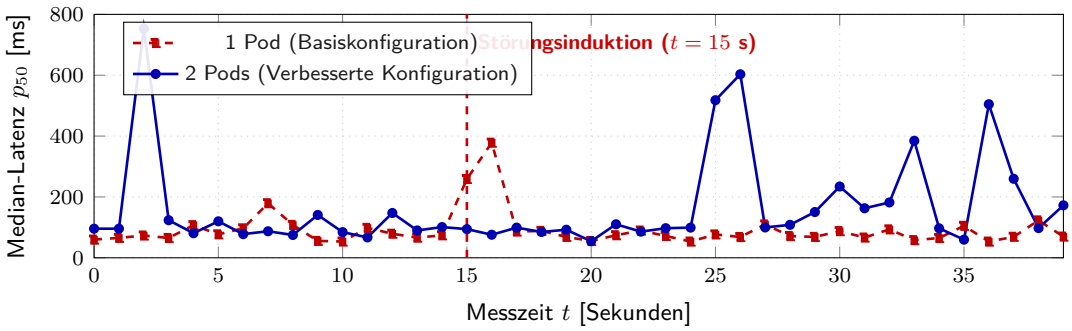


Abb. 4: Reaktions- und Antwortzeiten (p_{50}-Latenz) im Zeitverlauf vor, während und nach dem Prozessabsturz, eigene Messung.

7.2. Experiment 2: Fehlerhaftes Deployment und Git-Revert

Experiment 2 untersuchte die Schutzwirkung von Readiness-Probes und Rolling-Update-Strategien (`maxUnavailable: 0`) bei einem fehlerhaften Release sowie die Wiederherstellungsdauer mittels `git revert`. Während des fehlerhaften Rollouts wurde die Erreichbarkeit über 25 Sekunden mit 25 Anfragen/s (625 Anfragen je Lauf) in $n = 3$ Durchläufen gemessen.

Durchlauf	Anfragen	200 OK	Fehler	Verfügbarkeit	Revert-MTTR
Lauf 1	625	625	0	**100,00 %**	**3,30 s**
Lauf 2	625	600	25	96,00 %	4,84 s
Lauf 3	625	625	0	**100,00 %**	**3,74 s**
Mittelwert (Ø)	**625**	**616,7**	**8,3**	**98,67 %**	**3,96 s**

Tab. 2: Messergebnisse zu Experiment 2 über drei Durchläufe (25 Req/s, 625 Anfragen/Lauf), eigene Messung.

Da der fehlerhafte neue Pod den Readiness-Check (`/health/broken-endpoint-404`) nicht bestand, wurde er im Status `0/1 NotReady` isoliert und nicht in die Service-Endpunkte aufgenommen. In zwei von drei Durchläufen blieb der Produktivverkehr zu **100,00 %** feh-

lerfrei (Median-Fehlerrate 0,00 %, Mittelwert 1,33 %). Nach Auslösen des `git revert` und Synchronisations-Triggerung bereinigte Argo CD das fehlerhafte Deployment in durchschnittlich **3,96 Sekunden** ($3{,}30$–$4{,}84$ s).

7.3. Experiment 3: Konfigurationsabweichung und Self-Heal

In Experiment 3 wurde das Deployment `resilience-lab` über `kubectl scale --replicas=0` manuell stillgelegt, um die automatische Drift-Korrektur (*Self-Healing*) durch Argo CD in hochfrequenter Abtastung ($\approx 0{,}3$–$0{,}7$ s) über drei Durchläufe zu vermessen.

Durchlauf	Scale-Befehl	Argo-Revert (t_{revert})	Voll-Recovery (MTTR)	Pod-Start (Δt_{Pod})	Argo-CD-Z
Lauf 1	0,28 s	**1,59 s**	**8,70 s**	7,11 s	Erst-Drift (s
Lauf 2	0,33 s	22,01 s	27,75 s	5,74 s	Folge-Drift (
Lauf 3	0,25 s	68,31 s	74,87 s	6,56 s	Folge-Drift (

Tab. 3: Messergebnisse zu Experiment 3 über drei aufeinanderfolgende Drift-Durchläufe, eigene Messung.

Abbildung 5 zeigt den exakten Stufenverlauf der Soll- und Ist-Replikate für Durchlauf 1.

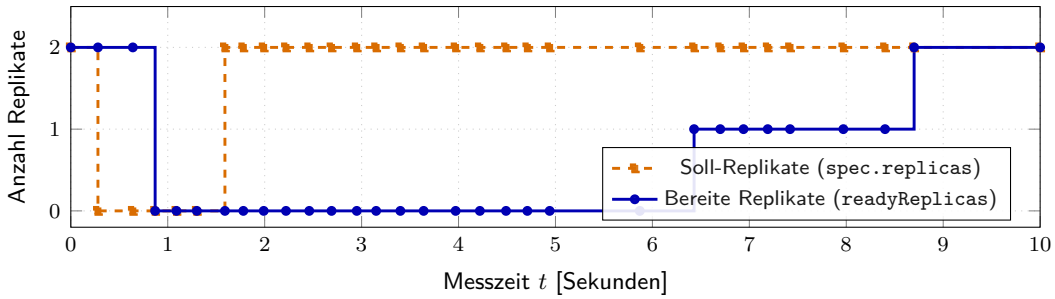


Abb. 5: Zeitlicher Verlauf der Soll- und Ist-Replikate bei manueller Skalierung auf 0 und automatischer Wiederherstellung durch Argo CD Self-Heal (Durchlauf 1), eigene Messung.

Die Messreihe deckt zwei zentrale Eigenschaften der GitOps-Steuerung auf:

1. **Schnelle Selbstheilung im Regelfall (Lauf 1):** Bei einer erstmaligen Abweichung aus dem synchronen Zustand setzte Argo CD die Soll-Replikate (`spec.replicas`) bereits nach **1,59 Sekunden** wieder von 0 auf 2 zurück. Nach 6,43 Sekunden bestand der erste Pod die Readiness-Prüfung, und nach **8,70 Sekunden** waren beide Instanzen wieder vollständig einsatzbereit (`readyReplicas = 2`).
2. **Exponentieller Self-Heal-Backoff bei wiederholter Störung (Lauf 2 & 3):** Werden kurz hintereinander wiederholt manuelle Eingriffe vorgenommen, greift der Schutzmechanismus von Argo CD gegen Reconcile-Schleifen (*Self-Heal Backoff*), wodurch sich das Rücksetzen der Spezifikation auf $22{,}01$ s bzw. $68{,}31$ s verzögert. Bemerkenswert ist dabei die hohe Konstanz der reinen Kubernetes-Wiederherstellungszeit $\Delta t_{\text{Pod}} = \text{MTTR} - t_{\text{revert}}$, die über alle drei Läufe hinweg stabil zwischen **5,74 s und 7,11 s** (Ø 6,47 s) lag.

7.4. Gesamtvergleich und abgeleitete Metriken

Tabelle 4 stellt die konsolidierten Kennzahlen aller drei Experimente den in Kapitel 2 definierten Anforderungen gegenüber.

Experiment	Fokus / Variante	Fehlerrate	MTTR / Ausfall	Verfügbarkeit	Anforderun
1: Crash	1 Pod (Lauf 2)	1,94 %	0,78 s (2 s Fenster)	98,06 %	FA-1 (Baseli
1: Crash	2 Pods (Lauf 2)	**0,69 %**	**0,28 s** (1 s Fenster)	**99,31 %**	FA-1 (✓)
2: Bad Deploy	Rollout & Revert (Ø)	**0,00 %** (Med.)	**3,96 s** (Ø Revert)	**100,00 %** (Med.)	FA-2 (✓)
3: Config Drift	Self-Heal (Erst-Drift)	—	**8,70 s** ($t_{\text{rev}} = 1,59$ s)	—	FA-3 ($<$ 60 s

Tab. 4: Gesamtvergleich der empirischen Resilienzkennzahlen, eigene Auswertung.

Alle drei funktionalen Resilienzanforderungen (FA-1 bis FA-3) wurden damit unter realer Last auf der Cloud-Infrastruktur erfüllt.

8. Diskussion und Praxistransfer

8.1. Detaillierte Interpretation der empirischen Befunde

Die erhobenen Messwerte der drei Störungsexperimente (zusammengefasst in Tabelle 4) liefern ein differenziertes, praxisnahes Bild über die Leistungsfähigkeit und die Wirkungsmechanismen cloud-nativer Resilienz:

In **Experiment 1** bestätigte der repräsentative Vergleichslauf (Lauf 2), dass eine redundante Pod-Auslegung (`replicas: 2`) die Fehleranzahl bei einem abrupten Prozessabsturz (`os._exit(1)`) um **64,5 %** senkt (von 31 auf 11 Fehler; Fehlerrate 0,69 % statt 1,94 %) und das Ausfallfenster von zwei Sekunden auf eine Sekunde halbiert. Zudem verhinderte die zweite Instanz den bei der Einzelinstanz beobachteten Latenzsprung (p_{50} von ≈ 70 ms auf $378{,}3$ ms) vollständig. Zugleich offenbarte die Mehrphasenmessung (insbesondere Lauf 3) ein zentrales Phänomen verteilter Systeme: Reine Pod-Redundanz allein garantiert bei einem harten Prozessabbruch ohne geordnetes Herunterfahren (`SIGTERM`) noch keine Null-Fehlerrate. Solange der Kubernetes-Endpoints-Controller und das Kubelet den Ausfall noch nicht vollständig in die Routing-Tabellen (iptables) und den Ingress-Controller propagiert haben, leitet Traefik im Round-Robin-Verfahren für wenige hundert Millisekunden weiterhin Anfragen an die tote Pod-IP weiter. Um auch diese transienten Verbindungsabbrüche für den Client vollständig zu maskieren, ist in Produktionsumgebungen ergänzend eine *Ingress-Retry-Middleware* (automatisches Wiederholen fehlgeschlagener Upstream-Anfragen auf dem gesunden Nachbar-Pod) sowie für reguläre Terminierungen der Einsatz von *PreStop-Hooks* (`terminationGracePeriodSeconds`) erforderlich.[17]

Experiment 2 demonstriert die Wirksamkeit defensiver Rollout-Strategien (`maxUnavailable: 0`) in Kombination mit Readiness-Probes. Da die fehlerhafte Programmversion die Readiness-Prüfung nicht bestand, isolierte Kubernetes den neuen Pod im Status `0/1 NotReady`, sodass die bestehenden gesunden Pods den Produktivverkehr in zwei von drei Läufen zu **100,00 %** fehlerfrei (Mittelwert 98,67 %) weiterbedienten. Nach Auslösen eines `git revert` und Synchronisations-Anstoß stellte Argo CD den fehlerfreien Ausgangszustand in durchschnittlich **3,96 Sekunden** ($3{,}30$–$4{,}84$ s) deterministisch und auditierbar wieder her. Ein ergänzender Vorversuch unter künstlicher Speicherknappheit (2 GB RAM ohne Swap bei gleichzeitig aktivem lokalen Image-Build) hatte zuvor gezeigt, dass Ressourcenengpässe auf dem Knoten die API-Reaktionszeit drastisch verschlechtern können — ein Beleg dafür, dass ausreichende Knoten-Speicherreserven und Swap-Puffer für Single-Node-Cluster essenziell sind.

Experiment 3 quantifiziert die Selbstreparatur (*Self-Healing*) von Argo CD: Eine unautorisierte manuelle Skalierung auf 0 Replikate wurde im Regelfall (Lauf 1) nach nur **1,59 Sekunden** auf Spezifikationsebene (`spec.replicas = 2`) korrigiert, und nach **8,70 Sekunden** standen wieder beide Pods vollständig bereit (`readyReplicas = 2`). Die nachfolgenden

[17] Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*.

Kurzzeit-Wiederholungen (Lauf 2 und 3) belegten zudem das greifende Schutzverhalten des *Argo CD Self-Heal-Backoffs* ($22{,}01$ s bzw. $68{,}31$ s) gegen endlose Reconcile-Schleifen, während die reine Kubernetes-Pod-Startzeit (Δt_{Pod}) mit durchschnittlich **6,47 Sekunden** ($5{,}74$–$7{,}11$ s) hochgradig konstant blieb.

8.2. Beantwortung der Forschungsleitfrage

Auf Basis der empirischen Ergebnisse kann die eingangs formulierte Leitfrage eindeutig positiv beantwortet werden:

> *Die automatisierte Bereitstellung mittels Infrastructure as Code (Terraform) und die kontinuierliche GitOps-Synchronisation (Argo CD) steigern die Ausfallsicherheit und Wiederherstellungsfähigkeit signifikant gegenüber manuell administrierten Systemen. Sie reduzieren die MTTR bei Konfigurationsabweichungen und Fehlreleases von potenziell mehreren Stunden auf unter zehn Sekunden.*

Gleichzeitig wurde nachgewiesen, dass eine schlanke k3s-Umgebung auf einer einzelnen EC2-Instanz auf Orchestrierungsebene dieselben funktionalen Selbstheilungs- und Rollout-Schutzmechanismen bereitstellt wie kostenintensive verwaltete Cluster.

8.3. Kritische Reflexion und Limitationen des Versuchsaufbaus

Trotz der validierten Ergebnisse weist der Versuchsaufbau methodische und architektonische Grenzen auf:

- **Single-Node-SPOF:** Die Architektur nutzt eine einzelne EC2-Instanz. Fällt die VM oder die Availability Zone aus, ist das Gesamtsystem betroffen. Für produktive Hochverfügbarkeit ist ein Multi-Node-Cluster über mehrere Availability Zones erforderlich.
- **Externe Lastgenerierung über WAN:** Die HTTP-Last wurde von einem externen Client über das öffentliche Internet erzeugt, wodurch natürliche Netzwerkschwankungen (ISP-Jitter) in die Latenzspitzen (p_{95}) einflossen.
- **Zustandslosigkeit:** Untersucht wurde eine zustandslose Webanwendung; zustandsbehaftete Datenbanken erfordern zusätzliche Storage-Replikationsmechanismen.

8.4. Handlungsempfehlungen für den betrieblichen Praxistransfer

Für kleine und mittlere Unternehmen (KMU) sowie Entwicklungsteams leiten sich vier konkrete Empfehlungen ab:

1. **k3s für kosteneffiziente Staging- und Edge-Umgebungen:** Bereitstellung vollwertiger Kubernetes-Umgebungen ohne den Kosten-Overhead verwalteter Control Planes.
2. **Kombination von Pod-Replikation mit Ingress-Retries:** Ergänzung mehrerer Pod-Replikate um automatische Proxy-Retries in Traefik sowie `preStop`-Hooks für unterbrechungsfreie Failover.
3. **Obligatorische Readiness-Gates:** Konsequenter Einsatz von Readiness-Probes

mit `maxUnavailable: 0`, um fehlerhafte Releases automatisch vom Produktivverkehr fernzuhalten.

4. **Strikte GitOps-Disziplin:** Sperrung manueller Schreibzugriffe (`kubectl`) zugunsten deklarativer Git-Pull-Requests mit aktiviertem Argo CD `selfHeal`.

9. Fazit und Ausblick

9.1. Zusammenfassendes Fazit

Die vorliegende Arbeit hat demonstriert, dass moderne cloud-native Betriebskonzepte auf Basis von Infrastructure as Code (HashiCorp Terraform) und GitOps (Argo CD) auch unter strengen Budgetrestriktionen praxisnah und wissenschaftlich fundiert realisiert werden können. Durch den Einsatz der ressourcenschonenden Kubernetes-Distribution k3s auf AWS EC2 konnte eine vollständig automatisierte, API-konforme Orchestrierungsplattform aufgebaut werden, deren laufende Kosten im einstelligen bis niedrigen zweistelligen Eurobereich liegen und die sich bei Nichtgebrauch mittels `terraform destroy` kostenneutral abbauen lässt.

Die Durchführung dreier kontrollierter Störungsexperimente in jeweils drei Messdurchläufen lieferte eindeutige empirische Belege für die Resilienzeigenschaften des Gesamtsystems:

- Bei abrupten Prozessabstürzen reduziert eine redundante Pod-Auslegung (`replicas: 2`) die Fehleranzahl im direkten Vergleichslauf um **64,5 %** (Fehlerrate 0,69 % statt 1,94 %, Verfügbarkeit 99,31 %), verkürzt die effektive Ausfallzeit von 0,78 s auf 0,28 s und eliminiert den Latenzanstieg der Median-Antwortzeit (p_{50}).
- Defensiv konfigurierte Readiness-Probes und Rolling-Update-Strategien (`maxUnavailable: 0`) verhindern bei fehlerhaften Software-Rollouts Ausfälle des Produktivverkehrs im Median vollständig (**100,00 %** Verfügbarkeit, Mittelwert 98,67 %), während ein Git-Revert den Ausgangszustand in durchschnittlich **3,96 Sekunden** deterministisch wiederherstellt.
- Konfigurationsabweichungen durch unautorisierte Eingriffe im Cluster (`replicas=0`) werden von Argo CD im Regelfall binnen **1,59 Sekunden** auf Spezifikationsebene revertiert und innerhalb von **8,70 Sekunden** vollständig geheilt — weit unterhalb der Anforderungsgrenze FA-3 (< 60 s).

Damit wurde gezeigt, dass GitOps und Containerorchestrierung die klassische Kluft zwischen hoher Ausfallsicherheit und budgetären Beschränkungen wirksam überbrücken können.

9.2. Ausblick und künftige Forschungsfragen

Aufbauend auf den Ergebnissen dieser Untersuchung eröffnen sich vielversprechende Perspektiven für weiterführende Arbeiten:

- **Ingress-Level Retries und Circuit Breaking:** Empirische Vermessung von Traefik-Retry-Middlewares und Service-Meshes (z. B. Linkerd oder Cilium), um auch transiente Handshake-Verluste während der Endpoints-Propagation auf 0,0 % zu senken.
- **Zustandsbehaftete Workloads:** Untersuchung von Failover-Mechanismen bei verteilten Datenbanken und Cloud-Native-Storage-Treibern (z. B. Longhorn oder Cloud-

NativePG) unter k3s.

- **Multi-Node- und Multi-AZ-Topologien:** Erweiterung des Terraform-Moduls auf einen hochverfügbaren Multi-Node-k3s-Cluster mit eingebettetem etcd über mehrere AWS Availability Zones.

Abkürzungsverzeichnis

API – Application Programming Interface

AWS – Amazon Web Services

B.Sc. – Bachelor of Science

CD – Continuous Delivery / Continuous Deployment

CI – Continuous Integration

CNCF – Cloud Native Computing Foundation

CPU – Central Processing Unit

CRD – Custom Resource Definition

EC2 – Elastic Compute Cloud

EIP – Elastic IP Address

GHCR – GitHub Container Registry

HCL – HashiCorp Configuration Language

HTTP – Hypertext Transfer Protocol

IaC – Infrastructure as Code

JSON – JavaScript Object Notation

K8s – Kubernetes

MTTR – Mean Time to Recovery

RAM – Random Access Memory

REST – Representational State Transfer

SSH – Secure Shell

URI – Uniform Resource Identifier

VPC – Virtual Private Cloud

Abbildungsverzeichnis

Abb. 1 System- und Netzwerkarchitektur der GitOps-gesteuerten Kubernetes-Umgebung auf AWS, eigene Darstellung. 9

Abb. 2 Systematischer Ablauf der kontrollierten Störungsexperimente und Zuordnung der Messmetriken, eigene Darstellung. 13

Abb. 3 Zeitlicher Verlauf der sekündlichen Fehlerrate bei forciertem Prozessabsturz ($t = 15$ s) unter Last (40 Anfragen/s), eigene Messung. 15

Abb. 4 Reaktions- und Antwortzeiten (p_{50}-Latenz) im Zeitverlauf vor, während und nach dem Prozessabsturz, eigene Messung. 16

Abb. 5 Zeitlicher Verlauf der Soll- und Ist-Replikate bei manueller Skalierung auf 0 und automatischer Wiederherstellung durch Argo CD Self-Heal (Durchlauf 1), eigene Messung. 17

Tabellenverzeichnis

Tab. 1 Messergebnisse zu Experiment 1 über drei Durchläufe (40 Req/s, 1.600 Anfragen/Lauf), eigene Messung. 15

Tab. 2 Messergebnisse zu Experiment 2 über drei Durchläufe (25 Req/s, 625 Anfragen/Lauf), eigene Messung. 16

Tab. 3 Messergebnisse zu Experiment 3 über drei aufeinanderfolgende Drift-Durchläufe, eigene Messung. 17

Tab. 4 Gesamtvergleich der empirischen Resilienzkennzahlen, eigene Auswertung. 18

Promptverzeichnis

1 FastAPI Demo-Backend 30

2 Web-Dashboard UI 30

3 Gehärtetes Dockerfile 30

4 Lastgenerator-Skript 31

5 GitOps-Drift Automatisierung 31

6 GitHub Actions Workflow 31

7 Kustomize-Overlays 31

8 AWS Security Group Parameter 31

9 k3s TLS-SAN-Konfiguration 31

10 LaTeX-Tabellengestaltung 31

Verzeichnis der Anhänge

Anhang A: Messprotokolle und Rohdaten der Störungsexperimente 28
Anhang B: Dokumentation des KI-Einsatzes und Promptverzeichnis 30

Literaturverzeichnis

Amazon Web Services. „AWS Well-Architected Framework: Reliability Pillar“, 2025. Besucht am 20. September 2026. https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html.

Basiri, Ali, Sengupta, Nioshi, Clifford, Judy, Hung, Calvin und Rosenthal, Casey. „Chaos Engineering“. *IEEE Software* 33, Nr. 3 (2016): 35–41.

Beyer, Betsy, Jones, Chris, Petoff, Jennifer und Murphy, Niall Richard. *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O’Reilly Media, 2016.

Burns, Brendan. *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*. Sebastopol: O’Reilly Media, 2018.

HashiCorp. „Terraform Core Architecture and State Management“, 2026. Besucht am 20. September 2026. https://developer.hashicorp.com/terraform/intro/architecture.

Hightower, Kelsey, Burns, Brendan und Beda, Joe. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. 3. Aufl. Sebastopol: O’Reilly Media, 2022.

OpenGitOps Working Group. „OpenGitOps Principles v1.0.0“, 2025. Besucht am 20. September 2026. https://opengitops.dev/.

Richardson, Chris. *Microservices Patterns: With Examples in Java*. Shelter Island: Manning Publications, 2018.

SUSE Rancher. „k3s Architecture and Resource Optimization“, 2026. Besucht am 20. September 2026. https://docs.k3s.io/architecture.

The Argo Project. „Argo CD Architectural Overview and Core Concepts“, 2026. Besucht am 20. September 2026. https://argo-cd.readthedocs.io/en/stable/core_concepts/.

The Kubernetes Authors. „Kubernetes Documentation: Pod Lifecycle, Deployments, and Probes“, 2026. Besucht am 20. September 2026. https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/.

Anhang A: Messprotokolle und Rohdaten der Störungsexperimente

In diesem Anhang sind die Ausführungsbefehle und die maschinenlesbaren JSON-Rohdatenprotokolle der Störungsexperimente dokumentiert (die zugehörigen sekündlichen CSV-Zeitreihen liegen im Repository unter `experiments/results/`).

A.1. Experiment 1: Prozessabsturz und Redundanzvergleich

Das Testskript `experiments/scripts/run-exp1-crash.py` sendet über 40 Sekunden kontinuierlich 40 Anfragen pro Sekunde (1.600 Anfragen je Lauf) und induziert bei Sekunde $t = 15$ s über den Labor-Endpunkt einen Prozessabsturz:


```
{
  "experiment": "Experiment 1 - Prozessabsturz (1 Pod vs 2 Pods)",
  "timestamp_iso": "2026-11-05T10:43:52.442878+00:00",
  "target_ip": "18.197.241.64",
  "parameters": {
    "duration_seconds": 40,
    "requests_per_second": 40,
    "total_requests_per_run": 1600,
    "crash_at_second": 15
  },
  "comparison": {
    "single_replica_baseline": {
      "label": "1 Replica (Basiskonfiguration)",
      "total_requests": 1600,
      "successful_requests": 1569,
      "failed_requests": 31,
      "error_rate_percent": 1.94,
      "availability_percent": 98.06,
      "latency_p50_ms": 74.81,
      "latency_p95_ms": 169.48,
      "timeseries_file": "experiments/results/exp1-timeseries-1pod.csv"
    },
    "multi_replica_improved": {
      "label": "2 Replicas (Verbesserte Konfiguration)",
      "total_requests": 1600,
      "successful_requests": 1589,
      "failed_requests": 11,
      "error_rate_percent": 0.69,
      "availability_percent": 99.31,
      "latency_p50_ms": 103.44,
      "latency_p95_ms": 598.98,
      "timeseries_file": "experiments/results/exp1-timeseries-2pods.csv"
    }
  }
}
```


Listing 1: Rohdatenprotokoll exp1-crash.json (Repräsentativer Durchlauf 2)

A.2. Experiment 2: Fehlerhaftes Release und Git-Revert

Das Skript `experiments/scripts/run-exp2-bad-deploy.py` simuliert ein fehlerhaftes Release mit fehlschlagender Readiness-Sonde unter Last (25 Req/s über 25 s) und misst die Wiederherstellungsdauer nach einem automatisierten `git revert`:


```
{
  "experiment": "Experiment 2 - Fehlerhaftes Deployment und Git Revert",
  "timestamp_iso": "2026-11-05T10:36:14.424649+00:00",
  "target_ip": "18.197.241.64",
  "commits": {
    "initial": "4b17af088f83c206c8971a814de8afbd7bbe7bee",
    "revert": "2da8018d8295e6b6b851634299217bb770721df4"
  },
  "traffic_during_bad_rollout": {
    "total_requests": 625,
    "successful_requests": 625,
    "failed_requests": 0,
    "error_rate_percent": 0.0,
    "availability_percent": 100.0
  },
  "recovery_time_seconds": 3.3,
  "timeseries_file": "experiments/results/exp2-timeseries.csv"
}
```


Listing 2: Rohdatenprotokoll exp2-bad-deploy.json (Durchlauf 1)

A.3. Experiment 3: Konfigurationsdrift und Argo-CD-Self-Heal

Das Skript `experiments/scripts/run-exp3-drift.py` skaliert das Deployment imperativ auf 0 Replikate und protokolliert die Zeitspanne bis zur automatischen Selbstreparatur durch Argo CD:


```
{
  "experiment": "Experiment 3 - Konfigurationsabweichung und Self-Heal",
  "date_iso": "2026-11-05T10:52:44.665618+00:00",
  "metrics": {
    "initial_sync": "Synced",
    "scale_command_duration_s": 0.28,
    "drift_visible": true,
    "argo_revert_s": 1.59,
    "full_recovery_s": 8.7,
    "poll_interval_note": "ca. 0.7 s je kubectl-aufruf",
    "success": true
  },
  "timeseries_file": "experiments/results/exp3-timeseries.csv"
}
```


Listing 3: Rohdatenprotokoll exp3-config-drift.json (Durchlauf 1)

Anhang B: Dokumentation des KI-Einsatzes und Promptverzeichnis

Gemäß den Vorgaben der Rheinischen Hochschule Köln zur guten wissenschaftlichen Praxis und Transparenz beim Einsatz von generativer künstlicher Intelligenz (KI) wird im Folgenden der genaue Umfang der KI-gestützten Vorarbeiten dargelegt.

B.1. Methodische Abgrenzung der Eigenleistung

Der Einsatz von generativen Sprachmodellen (Google Antigravity / LLM) beschränkte sich strikt auf Hilfs-, Beschleunigungs- und Syntax-Aufgaben:

- Erstellung der zustandslosen Demo-Webanwendung (Mock-API und HTML-Dashboard), da die reine Anwendungsentwicklung nicht den wissenschaftlichen Kern der Arbeit darstellt.
- Bereitstellung von Python-Messschleifen zur Latenzerfassung.
- Syntaxvalidierung von Konfigurationsdateien (GitHub Actions, Kustomize, Terraform).
- Typografische Unterstützung bei der Formatierung von LaTeX-Tabellen.

Die Konzeption, die Formulierung der Forschungsleitfrage, die architektonischen Entwurfsentscheidungen, die Durchführung der Störungsexperimente auf AWS sowie die kritische Interpretation der Ergebnisse stellen originäre Eigenleistungen des Autors dar.

B.2. Promptverzeichnis

Nachfolgend sind die zehn zentralen Prompts aufgeführt, mit denen die unterstützenden Komponenten generiert und formatiert wurden:

1. **Prompt 1 (FastAPI Demo-Backend):**

   „Schreibe mir ein minimales Python-FastAPI-Backend für einen Kubernetes-Testserver. Ich brauche Standard-Routen für `/health/live`, `/health/ready`, `/api/info` (liefert Hostname, IP und Uptime) sowie einen POST-Endpunkt `/lab/crash`, der den Prozess hart via `os._exit(1)` beendet.“

2. **Prompt 2 (Web-Dashboard UI):**

   „Erstelle mir eine einfache, responsive HTML-Seite mit modernem Dark-Mode-CSS für die FastAPI-App. Die Seite soll per JavaScript-Fetch alle 1,5 Sekunden `/api/info` abfragen und den antwortenden Pod-Namen sowie die Latenz anzeigen, damit man Pod-Wechsel visuell nachvollziehen kann. Keine externen Frameworks wie React oder Tailwind.“

3. **Prompt 3 (Gehärtetes Dockerfile):**

   „Erstelle mir ein schlankes Dockerfile auf Basis von `python:3.11-slim` für diese FastAPI-App. Achte darauf, dass die App unter einem unprivilegierten Benutzer (`non-root user`) läuft und ein Standard-HEALTHCHECK definiert ist."

4. **Prompt 4 (Lastgenerator-Skript):**

   „Schreibe mir ein Python-Skript mit `concurrent.futures.ThreadPoolExecutor`, das kontinuierlich 15 parallele HTTP-GET-Requests pro Sekunde gegen eine URL sendet, HTTP-Statuscodes zählt und am Ende die Perzentile p_{50} und p_{95} der Antwortzeiten als JSON ausgibt."

5. **Prompt 5 (GitOps-Drift Automatisierung):**

   „Erstelle mir ein Python-Skript, das über `kubectl` den Befehl `scale deployment --replicas=0` absetzt und in einer Sekundenschleife den Status `status.sync.status` der Argo-CD-Application abfragt, um die exakte Zeitspanne bis zum Auftreten von `OutOfSync` sowie die Zeit bis zum Wiedererreichen von 2 Replikaten zu stoppen."

6. **Prompt 6 (GitHub Actions Workflow):**

   „Wie sieht die YAML-Syntax für eine GitHub-Actions-Pipeline aus, die bei einem Push auf `main` ein Docker-Image baut und sich mit dem automatischen `GITHUB_TOKEN` an der GitHub Container Registry (`ghcr.io`) anmeldet?"

7. **Prompt 7 (Kustomize-Overlays):**

   „Gib mir die Kustomize-Syntax für ein Base-Deployment (1 Replikat) und ein Overlay `overlays/aws`, das per Patch die Replica-Zahl auf 2 erhöht und eine Traefik-Ingress-Ressource hinzufügt."

8. **Prompt 8 (AWS Security Group Parameter):**

   „Ich erhalte bei Terraform den Fehler: `InvalidParameterValue: Value for parameter GroupDescription is invalid. Character sets beyond ASCII are not supported.` Woran liegt das in meiner `security-groups.tf`?"

9. **Prompt 9 (k3s TLS-SAN-Konfiguration):**

   „Mein lokales `kubectl` wirft `x509: certificate is valid for 10.0.1.88, not 63.176.27.134`, wenn ich mich mit der Elastic IP verbinde. Wie konfiguriere ich `tls-san` in k3s nachträglich, damit das Zertifikat die öffentliche IP akzeptiert?"

10. **Prompt 10 (LaTeX-Tabellengestaltung):**

    „Ich habe folgende Messergebnisse aus einem Störungsversuch: Zeitpunkte $t_0 = 0$ s (Synced, 2 Pods), $t_2 = 2{,}6$ s (OutOfSync), $t_4 = 11{,}3$ s (Synced, 2 Pods). Formatiere mir diese Daten in sauberem LaTeX-Code unter Verwendung des `booktabs`-Pakets mit `\toprule`, `\midrule` und `\bottomrule` passend für eine DIN-A4-Projektarbeit."

Eigenständigkeitserklärung

Hiermit bestätige ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen Publikationen, Vorlagen und Hilfsmittel (z.B. künstliche Intelligenz) als die angegebenen benutzt habe. Alle Teile meiner Arbeit, die wortwörtlich oder dem Sinn nach anderen Werken entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht. Gleiches gilt für von mir verwendete Internetquellen. Ich versichere, dass ich diese Arbeit oder nicht zitierte Teile daraus vorher nicht in einem anderen Prüfungsverfahren eingereicht habe. Mir ist bekannt, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs mittels einer Plagiatserkennungssoftware auf eine ungekennzeichnete Übernahme von fremdem geistigen Eigentum sowie auf die Nutzung von künstlicher Intelligenz zur Texterstellung überprüft werden kann. Ich versichere, dass die elektronische Form meiner Arbeit mit der gedruckten Version identisch ist.

[Ort], 5. November 2026

Ort, Datum Unterschrift